Laboratorio 4 — Despliegue del sistema en la nube

Base de datos, caché y almacenamiento de archivos en producción

Sistemas Distribuidos · UPTC Sogamoso · Octubre de 2026 · Documento oficial de entrega y sustentación

Léanlo completo antes de preguntar. Este documento define qué se entrega, cómo se evalúa y qué condiciones no son negociables. Cualquier duda se resuelve primero releyendo la sección correspondiente.

1. Objetivo

Llevar el sistema académico a la nube. Partiendo del Laboratorio 3 (Gateway y tres módulos), cada grupo debe **empaquetar todos sus componentes en contenedores y publicarlos en la nube de su proveedor asignado**, funcionando de extremo a extremo con:

- **Bases de datos administradas** (una por módulo),
- **caché compartido** (Redis administrado) en el Gateway,
- **almacenamiento de objetos** para fotografías de estudiantes y PDFs de materias,
- **autenticación con roles** y un **frontend** mínimo que consume el sistema.

Es la primera vez que el sistema deja de correr en su computador. Lo que construyeron en la Expo de nube (servicios, configuración y costos) se convierte ahora en un despliegue real, con costos reales.

Condición de entrega: el sistema debe estar **publicado y accesible en la nube del proveedor del grupo, por una URL pública con HTTPS**. Una entrega que solo funcione en el computador de los integrantes se considera incompleta (ver sección 8).

2. Modalidad, grupos y proveedores

Grupal. Los mismos grupos y el **mismo proveedor** de la Expo de nube. Cada integrante es responsable de uno o más componentes concretos del sistema (código, contenedor o infraestructura), y esto se declara en la entrega y se verifica en la sustentación.

3. Punto de partida: lo que ya tenemos y lo que se agrega

Este laboratorio **no empieza de cero**. Reutiliza y extiende lo que cada grupo ya hizo:

Origen	Qué aporta al Laboratorio 4
Laboratorio 3	Gateway y tres módulos independientes (Estudiantes, Materias, Inscripciones), cada uno con su API REST, su base de datos y su Swagger; enrutamiento por prefijo de path y endpoint de composición GET `/api/estudiantes/{id}/detalle`.
Expo de nube	El perfilado de servicios de su proveedor y la estimación de costos de producción. Ahora se contrasta con lo que realmente aprovisionan.
Clase de almacenamiento y caché	Qué va en base de datos, qué en objetos y qué en caché; patrón cache-aside, TTL, invalidación y qué no cachear.

Lo **nuevo** que se agrega al sistema en este laboratorio:

Componente nuevo	Descripción corta
Módulo de Autenticación	Su propia API y su propia base de datos (usuarios, roles y credenciales). Emite tokens; el Gateway los valida.
Almacenamiento de archivos	Fotografías de estudiantes y PDFs con el contenido temático de cada materia, en almacenamiento de objetos.
Caché	Redis administrado, usado por el Gateway para el catálogo de Materias y consultas de lectura repetidas.
Frontend	Aplicación web mínima que consume el Gateway.
Despliegue en la nube	Una imagen por componente, publicadas en un registro y ejecutadas en la nube del proveedor.

4. Reglas que se mantienen

La **regla de independencia** del Laboratorio 3 sigue vigente y se verifica de nuevo:

- Cada módulo tiene **repositorio propio**, **base de datos propia** (sin foreign keys entre módulos) y **ciclo de vida propio**: se despliega, se reinicia y se actualiza sin tocar a los demás.
- Ningún módulo copia datos de otro: los obtiene por HTTP en tiempo de ejecución.

- Los archivos no se guardan en la base de datos: la base guarda solo la referencia al archivo (sección 5.2).

Una instancia de base de datos puede contener varias bases, siempre que cada módulo tenga su **base, su esquema y sus credenciales independientes**. Es una decisión de costo permitida; deben declararla y justificarla. Cuatro bases significa Estudiantes, Materias, Inscripciones y Autenticación.

5. Requisitos funcionales nuevos

5.1 Autenticación y roles

- Módulo de **Autenticación** como proceso independiente, con su base de datos (usuarios, roles, credenciales).
- `POST /api/auth/login` recibe credenciales y devuelve un **token de acceso firmado** (por ejemplo JWT) con vencimiento.
- Las **contraseñas se almacenan con hash** (BCrypt, Argon2 o equivalente). Nunca en texto claro.
- Al menos **dos roles** (por ejemplo `ADMIN` y `ESTUDIANTE`). Una regla de autorización por rol debe poder demostrarse (ejemplo: un estudiante solo puede consultar su propio `/detalle`; solo el administrador crea o borra materias).
- El **Gateway valida el token** (firma y vencimiento) antes de enrutar. Son públicas únicamente el login y los endpoints de salud. Sin token válido: `401`. Con token pero sin permiso: `403`.
- La **clave de firma** del token es un secreto: no puede estar en el repositorio ni en la imagen.

5.2 Archivos: fotografías y PDFs

- **Fotografía del estudiante:** el módulo Estudiantes permite subirla y reemplazarla (`multipart/form-data`). Solo imágenes JPG o PNG, máximo **2 MB**.
- **Contenido temático de la materia:** el módulo Materias permite subir y listar archivos PDF por materia. Máximo **10 MB** por archivo.
- Los archivos viven en el **almacenamiento de objetos** del proveedor, **no** en la base de datos ni en el sistema de archivos del contenedor (que es efímero).
- La base de datos puede guardar la URL del archivo pero como requisito no obligatorio pueden guardar solo los **metadatos**: clave del objeto, nombre original, tipo y tamaño, aunque esto no sería obligatorio solo si quieren llegar más allá.
- `GET /api/estudiantes/{id}/detalle` debe incluir la **referencia a la fotografía** del estudiante.

5.3 Caché en el Gateway

- Usar **Redis administrado en la nube** (no un caché en la memoria del proceso): debe ser compartido y sobrevivir al reinicio del Gateway.
- Patrón **cache-aside** con **TTL configurable** (por ejemplo entre 1 y 5 minutos).
- Cachear **como mínimo** las lecturas del catálogo de Materias (`GET /api/materias/...`), incluidas las consultas con paginación, orden y filtros. La clave debe incluir esos parámetros (y una versión, para poder invalidar).
- **Invalidación:** cualquier POST, PUT o DELETE sobre Materias invalida lo cacheado para ese recurso.
- **No cachear** los cupos de Inscripciones ni las escrituras. Deben poder explicar por qué.
- El Gateway agrega la cabecera de diagnóstico `X-Cache: HIT` o `X-Cache: MISS` en las respuestas cacheables.
- **Si Redis no está disponible, el sistema sigue funcionando** (más lento): el fallo del caché no puede tumbar las peticiones.
- Deben **medir** el efecto: tiempos de respuesta con y sin caché sobre al menos 50 peticiones, y el porcentaje de aciertos (hit ratio).

5.4 Frontend mínimo

- Aplicación web publicada en la nube (puede ser una SPA estática o una aplicación servida desde un contenedor).
- Pantallas mínimas: **inicio de sesión**; **listado de estudiantes** con su fotografía; **listado de materias** con descarga del PDF; **detalle de un estudiante** que consuma `/detalle`.
- El frontend habla **únicamente con el Gateway**, nunca directamente con los módulos ni con las bases de datos.

7. Requisitos de despliegue en la nube

Todo se despliega en **un único ambiente**, en el proveedor asignado al grupo. Se espera que el tamaño de los recursos sea **el mínimo que cumpla la función** (es un ambiente de demostración académica), pero el diseño y la configuración deben poder justificarse como en la Expo.

7.1 Cómputo

- Todos los componentes corren en la nube a partir de sus imágenes, en el servicio de cómputo que elijan: contenedores administrados o sin servidor (recomendado).
- El **Gateway y el Frontend** son los **únicos puntos de entrada públicos**, con **HTTPS** (basta con el dominio y el certificado que entrega el proveedor).

- Los módulos deben ser accesibles **solo desde el Gateway** cuando la plataforma lo permita (ingreso interno o red privada). Si la plataforma los expone públicamente por defecto, deben exigir autenticación y documentar la decisión.
- Configurar el **health check** del servicio del proveedor al menos para el Gateway y un módulo.

7.2 Bases de datos

- Servicio de **base de datos relacional administrada** del proveedor, con las **cuatro bases independientes** (sección 4).
- **Respaldos automáticos activados.**
- Acceso restringido: **no abierto a todo internet** (nada de reglas `0.0.0.0/0`); solo desde los componentes que las usan, mediante reglas de red o red privada. Si por costo o por limitaciones del plan no es posible, deben justificarlo por escrito.
- Usuario y contraseña distintos por módulo, guardados como secretos.

7.3 Caché

- Instancia de **Redis administrado** del proveedor, de tamaño mínimo que cumpla la función.
- Acceso restringido igual que las bases de datos; con contraseña o autenticación, y cifrado en tránsito si el plan lo permite.

7.4 Almacenamiento de objetos

- Un **bucket o contenedor privado** del servicio de almacenamiento de objetos del proveedor, con nombre y región declarados.

7.5 Seguridad y configuración

- Secretos (claves de firma, contraseñas, cadenas de conexión) en el **gestor de secretos** del proveedor o, como mínimo, en variables de entorno del servicio. **Nunca en git.**
- Cada componente con los **permisos mínimos** necesarios (por ejemplo, el módulo Estudiantes solo accede a su bucket y a su base).
- CORS configurado para permitir al frontend llamar al Gateway.

7.8 Créditos, presupuesto y buenas prácticas

- Cada grupo debe usar los **créditos educativos o el nivel gratuito** de su proveedor.
- Usen los tamaños mínimos que cumplan la función. Eliminen cualquier recurso de prueba que no se use.
- No dejen recursos sin vigilancia: un recurso olvidado puede consumir los créditos del grupo.
- **Mantengan el despliegue disponible el martes de la sustentación**.

8. Condiciones no negociables

Si alguna de estas condiciones no se cumple, la entrega se considera **incompleta** y no se sustenta hasta que se corrija:

- El sistema está **desplegado en la nube del proveedor del grupo**, con **URL pública HTTPS** para el frontend y el Gateway, y funciona durante la sustentación.
- Hay **base de datos, caché y almacenamiento de archivos administrados** realmente aprovisionados (no simulados ni solo en el computador).
- **No hay secretos en el repositorio ni en las imágenes.**
- Las imágenes están publicadas en el registro del proveedor y **el despliegue las usa**.
- Cada componente tiene un **responsable declarado** dentro del grupo.

9. Hitos y fechas

Hito	Fecha	Qué se verifica
Entrega y sustentación	Martes 3 de noviembre de 2026	Documento de entrega completo (sección 10) y sistema desplegado funcionando en vivo (sección 11).

El documento de entrega se sube a la carpeta de **Entregables** del curso en el Drive, en la subcarpeta **Laboratorio 4**, **antes de la sustentación**. Nombre sugerido: `lab4_grupoN_proveedor.md` (o `.pdf`), donde N es el número del grupo.

10. Qué deben entregar

Un documento por grupo, con enlaces a los repositorios, con lo siguiente:

- **Repositorios** de cada módulo, del Gateway, del frontend y del despliegue (`docker-compose.yml`, `.env.example`, instrucciones o scripts), con un README que explique la arquitectura y cómo levantar el entorno local.
- **URLs públicas** del frontend y del Gateway, y credenciales de **dos usuarios de prueba** (uno por rol) para el docente.
- **Tabla de recursos en la nube:** necesidad, servicio real aprovisionado, tipo (IaaS/PaaS/SaaS), configuración (región, tamaño, redundancia), con captura del grupo de recursos o proyecto.
- **Autenticación:** capturas del login, del token recibido, de una respuesta `401` y de una `403`.

- **Archivos:** captura de la subida de una foto y de un PDF; del objeto en el bucket privado; del acceso directo sin firma rechazado; y de la URL temporal funcionando y vencida. Captura de la base de datos mostrando que solo guarda metadatos.
- **Caché:** capturas de `X-Cache: MISS` y `HIT`, **tabla de tiempos** con y sin caché (≥ 50 peticiones), hit ratio, evidencia de la **invalidación** tras modificar una materia, y qué ocurre cuando Redis no está disponible.
- **Bases de datos:** captura de las cuatro bases independientes, respaldos activados y reglas de red.
- **Endpoint /detalle** funcionando en la nube, con la fotografía incluida.
- **Observaciones:** problemas encontrados al pasar de local a la nube y cómo los resolvieron.
- **Declaración de autoría:** qué integrante fue responsable de qué componente (código, contenedor, infraestructura).

11. Sustentación

Se hace **en vivo, sobre el sistema desplegado en la nube** (no en `localhost`), en la fecha indicada. Cualquier integrante puede recibir preguntas sobre cualquier componente.

Guion de la demostración (cada grupo la ejecuta en este orden):

1. Abrir el frontend por su **URL pública** e iniciar sesión con un usuario de cada rol; mostrar una diferencia de permisos.
2. Listar estudiantes y **subir una fotografía**; verla en el listado y en `/detalle`.
3. Subir y **descargar un PDF** de una materia.
4. Consultar el catálogo de Materias dos veces: **MISS y luego HIT**, con los tiempos; modificar una materia y mostrar la **invalidación**.
5. Mostrar las **bases de datos** (cuatro independientes, respaldos, reglas de red) y el **log** de una petición que atraviese Gateway y un módulo.
6. **Provocar un fallo** (detener el caché o un módulo) y explicar qué le pasa al sistema y qué debería pasar.

12. Rúbrica de evaluación

Criterio	Peso
Despliegue integral en la nube (todos los componentes funcionando, HTTPS, extremo a extremo)	20%
Bases de datos administradas, independientes, con respaldo y acceso restringido	10%
Caché: implementación, invalidación, medición y degradación ante fallo	15%
Almacenamiento de archivos: bucket privado, metadatos en BD, URLs firmadas, validaciones	15%

Criterio	Peso
Autenticación, roles y seguridad (secretos, red, permisos mínimos)	10%
Frontend funcional que consume el Gateway	10%
Documentación y evidencias	10%
Sustentación en vivo (todos los integrantes responden con propiedad)	10%

Anexo A — Qué debe ser público y qué privado

Componente	Visibilidad	Quién puede acceder
Frontend	Público (HTTPS)	Cualquier navegador
API Gateway	Público (HTTPS)	Cualquier cliente; las rutas protegidas exigen token
Autenticación, Estudiantes, Materias, Inscripciones	Privado (preferible)	Solo el Gateway; si la plataforma los expone, exigen autenticación
Bases de datos	Privado	Solo el módulo dueño de cada base
Redis (caché)	Privado	Solo el Gateway
Secretos	Privado	Gestor de secretos o variables del servicio; nunca git

Anexo B — Servicios equivalentes por proveedor (punto de partida)

No es una lista obligatoria: ya investigaron sus servicios en la Expo. Sirve para verificar que no falte nada. Confirmen siempre los nombres vigentes en la documentación de su proveedor.

Necesidad	AWS	Azure	Google Cloud	IBM Cloud
Registro de imágenes	ECR	Azure Container Registry	Artifact Registry	IBM Cloud Container Registry
Contenedores administrados o sin servidor	App Runner · ECS con Fargate	Container Apps	Cloud Run	Code Engine
Base de datos relacional administrada	RDS (PostgreSQL) · Aurora	Azure Database for PostgreSQL	Cloud SQL	Databases for PostgreSQL
Caché (Redis)	ElastiCache	Azure Managed Redis	Memorystore	Databases for Redis

Necesidad	AWS	Azure	Google Cloud	IBM Cloud
Almacenamiento de objetos	S3	Blob Storage	Cloud Storage	Cloud Object Storage
Gestor de secretos	Secrets Manager	Key Vault	Secret Manager	Secrets Manager
Logs	CloudWatch Logs	Azure Monitor (Log Analytics)	Cloud Logging	IBM Cloud Logs
Presupuestos y alertas	AWS Budgets	Cost Management (presupuestos)	Cloud Billing (presupuestos y alertas)	Notificaciones de gasto

Recordatorio: el objetivo no es solo que funcione, sino que cada integrante entienda por qué cada pieza está donde está, cuánto cuesta y qué pasa cuando algo falla.